COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S2	**SECTION:**	CPE21S2
DATE PERFORMED:	7/10/26	**DATE SUBMITTED:**	7/14/26
NAME:	Niel Vincent B. Condino	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1 **Basic C++ Programming**

1. PROCEDURE OUTPUT

Table 1-1.-1 C++ Structure Code for ILO 1

Header File		`#include <iostream>`
Global Declaration Section		

Class Declaration and Method Definition Section		`int sumOfTwoNum(int num1, int num2);`
		`bool isGreaterThan(float num1,float num2);`
		`bool boolCompare(bool bool1,bool bool2);`

Main Function		`int main(){`
		`  std::cout << "1+1 =" << sumOfTwoNum(1,1) << "\n";`
		`  std::string isBigger = isGreaterThan(3,4) ? " True": "False";`
		`  std::cout << "3 is greater than 4:" << isBigger << "\n";`
		`  std::cout << boolCompare(true,false);`
		`  return 0;`
		`}`

Method Definition		`int sumOfTwoNum(int num1,int num2){`
		`  return num1 + num2;`
		`}`
		
		`//bool parameters turn non-zero values to true bool and zero values turn false`

		`bool isGreaterThan(float num1,float num2){`
		`  return num1 > num2;`
		`}`
		
		`bool boolCompare(bool bool1,bool bool2){`
		`  std::cout << "using and: "<< (bool1 && bool2) << std::endl;`
		`  std::cout << "using or: " << (bool1 || bool2) << std::endl;`
		`  std::cout << "using not on bool1: " << !bool1 << std::endl;`
		`  std::cout << "using not on bool2: " << !bool2 << std::endl;`
		`  return true;`
		`}`

Output		`❯ cd "/home/nielc/Downloads/HW # 1.1/" && g++ main.cpp -o main && "/home/nielc/Downloads/HW # 1.1/"main`
		`1+1 =2`
		`3 is greater than 4:False`
		`using and: 0`
		`using or: 1`
		`using not on bool1: 0`
		`using not on bool2: 1`
		`1⏎`

Observation & Comments:

The procedure is easy enough to do. The first and second tasks are simple enough. The first one just adds, and the second just compares 2 integers. The third one is the one I find pretty unclear on what it wants to achieve at first; however, after the explanation, it became clear to me that it is very simple. It just applies the 3 logical operators in C++ which are: AND, OR, and NOT, and returns true at the end. Unlike in Python, C++ needs data types declared at compile time. Because I set the parameters to have a data type of bool, it will turn any non-zero numbers into the bool true, and zero will be turned to false.

Table 1-1-2. C++ Structure Code for ILO2

Header File	`#include <iostream>`

Declaration Section	
Method Definition Section	
Main Function	

```
class Triangle{
   private:
       double totalAngle,angleA,angleB,angleC;
   public:
       Triangle(double A, double B, double C);
       void setAngles(double A, double B, double
C);
       void setSides(double side1,double
side2,double side3);
       const bool validateTriangle();
};
```

```
int main(){
   Triangle set1(40, 30, 110,20,20,20);

   std::cout << "area is " << set1.computeArea() <<
"\n";
   std::cout << "perimeter is " <<
set1.computePerimeter() << "\n";
   std::cout << "is a " << set1.checkTypeByAngle()
<< " triangle\n";

   if(set1.validateTriangle()){
       std::cout << "The shape is a valid
triangle.\n";
   }
   else {
       std::cout << "The shape is NOT a valid
triangle.\n";
   }

   return 0;
}
```

Method Definition	`Triangle::Triangle(double A, double B, double C) {`
	`  angleA = A;`
	`  angleB = B;`
	`  angleC = C;`
	
	`  totalAngle = A+B+C;`
	`}`
	
	`void Triangle::setAngles(double A, double B, double C) {`
	`  angleA = A;`
	`  angleB = B;`
	`  angleC = C;`
	`  totalAngle = A+B+C;`
	`}`
	
	`const bool Triangle::validateTriangle() {`
	`  return (totalAngle <= 180);`
	`}`

Output	`> cd "/home/nielc/Downloads/HW # 1.1/" && g++ triangle.cpp -o triangle && "/home/nielc/Downloads/HW # 1.1/"triangle` `The shape is a valid triangle.`

Table 1-2. ILO B output observations and comments.

OOP in C++ is mostly the same as the ones of Python, because overall it is the same concept. The concepts of OOP in Python that I learned last semester still apply to this language. The difference I noticed is the syntax. To declare attributes in C++, we use the keywords private, public, or protected based on what type of attribute we want to create. Whereas in Python, the prefix "_" for protected or "__" for private is used to label an attribute as private. Also, unlike in Python, setting attributes is also disconnected from the constructor: which for python is__init__(). The constructor is also different. For python, the constructor is a method named __init__() whereas in C++, the constructor's syntax is *ClassName::ClassName()*. I also noticed that in C++, it does not use self as a first parameter, and you can go straight to the attributes.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#include <iostream>

void swapVar(int &var1,int &var2);

int main(){
   int a = 1;
   int b = 4;
   std::cout << "a is " << a << " and b is " << b << '\n';
   swapVar(a,b);
   std::cout << "a is "<< a << " and b is " << b << '\n';
   return 0;
}

void swapVar(int &var1,int &var2){
   int temp = 0;
   temp = var1;
   var1= var2;
   var2 = temp;
   std::cout<< "Swapped!";
}
```

Figure 1:C++ program that swaps the two numbers in different variables

This program is relatively simple. The algorithm used for swapping variables is very easy to understand. It utilizes a temporary variable to store the value of one of the variables. The first variable's value is stored in the temporary variable before being swapped with the second variable's value. The second variable then gets the value of the first variable from the temporary value created. The parameters used are references so that the original variables' values can be changed by the function.

```
~/Downloads/HW # 1.1
❯ cd "/home/nielc/Downloads/HW # 1.1/" && g++ swapvar.cpp -o swapvar &&
"/home/nielc/Downloads/HW # 1.1/"swapvar
a is 1 and b is 4
Swapped!
a is 4 and b is 1
```

Figure 1.1: Output of the code from Figure 1

```cpp
#include <iostream>

double changeKtoF(double inputTemp);

int main(){
   double input;
   std::cout << "Kelvin to Fahrenheit
converter\n----------------------------------------\n";
   std::cout << "Input temperature in Kelvin: ";
   std::cin >> input;
   double temp = changeKtoF(input);
   std::cout << input << " K in Fahrenheit is " << temp << " F";
   return 0;
}

double changeKtoF(double inputTemp){
   return ((inputTemp - 273) * 9/5) + 32;
}
```

Figure 2: C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

The function here is also simple. It accepts a Kelvin temperature input, which uses a double data type for precision. Using the formula for converting Kelvin to Celsius (Kelvin - 273) and the Celsius-to-Fahrenheit formula (Celsius x 1.8 + 32) to convert Kelvin to Fahrenheit. The program then utilizes this function by asking the user for an input Kelvin temperature that it then converts to Fahrenheit.

```
❯ cd "/home/nielc/Downloads/HW # 1.1/" && g++ tempchange.cpp -o
tempchange && "/home/nielc/Downloads/HW # 1.1/"tempchange
Kelvin to Fahrenheit converter
----------------------------------------
Input temperature in Kelvin: 273
273 K in Fahrenheit is 32 F⏎
```

Figure 2.1: Output of the code from Figure 2

```cpp
#include <iostream>
#include <cmath>

float calcDistance(float x1,float x2, float y1, float y2);

int main(){
   std::cout << "Enter points\n";
   float x1,x2,y1,y2;
   std::cout << "Enter the first point in the x axis:";
   std::cin >> x1;
   std::cout << "Enter the second point in the x axis:";
   std::cin >> x2;
   std::cout << "Enter the first point in the y axis:";
   std::cin >> y1;
   std::cout << "Enter the second point in the y axis:";
   std::cin >> y2;

   std::cout << "Distance is " << calcDistance(x1,x2,y1,y2);
}
float calcDistance(float x1,float x2, float y1, float y2){
   return (sqrt(pow(x1-x2,2)+pow(y1-y2,2)));
}
```

Figure 3:C++ program that has a function that will calculate the distance between two points

The code above is also a simple program that calculates the distance between 2 points on a 2D plane. The function takes in 4 parameters: the x-coordinate of the points and the y-coordinate of the points. From this information, the function calculates the distance using the distance formula. To achieve this computation, the pow() and sqrt() function from the cmath library is used.

```
❯ cd "/home/nielc/Downloads/HW # 1.1/" && g++ distanceformula.cpp -o
distanceformula && "/home/nielc/Downloads/HW # 1.1/"distanceformula
Enter points
Enter the first point in the x axis:0
Enter the second point in the x axis:4
Enter the first point in the y axis:0
Enter the second point in the y axis:7
Distance is 8.06226 units⏎
```

```cpp
#include <iostream>
#include <cmath>

class Triangle{
   private:
       double totalAngle,angleA,angleB,angleC,sideA,sideB,sideC;
   public:
       Triangle(double A, double B, double C,double side1,double
side2,double side3);
       void setAngles(double A, double B, double C);
       void setSides(double side1,double side2,double side3);
       const bool validateTriangle();
       const std::string checkTypeByAngle();
       const double computePerimeter();
       const double computeArea();
};

int main(){
   Triangle set1(40, 30, 110,20,20,20);

   std::cout << "area is " << set1.computeArea() << "\n";
   std::cout << "perimeter is " << set1.computePerimeter() << "\n";
   std::cout << "is a " << set1.checkTypeByAngle() << " triangle\n";

   if(set1.validateTriangle()){
       std::cout << "The shape is a valid triangle.\n";
   }
   else {
       std::cout << "The shape is NOT a valid triangle.\n";
   }

   return 0;
}

Triangle::Triangle(double A, double B, double C,double side1,double
```

```
side2,double side3) {
   angleA = A;
   angleB = B;
   angleC = C;
   sideA = side1;
   sideB = side2;
   sideC = side3;

   totalAngle = A+B+C;
}

void Triangle::setAngles(double A, double B, double C) {
   angleA = A;
   angleB = B;
   angleC = C;
   totalAngle = A+B+C;
}

const bool Triangle::validateTriangle() {
   return (totalAngle <= 180);
}

const double Triangle::computePerimeter(){
   return sideA + sideB + sideC;
}

const double Triangle::computeArea(){
   double semiPerimeter = Triangle::computePerimeter()/2.0;
   return
sqrt(semiPerimeter*(semiPerimeter-sideA)*(semiPerimeter-sideB)*(semiP
erimeter-sideC));//Heron's Formula
}

const std::string Triangle::checkTypeByAngle(){
   if (angleA > 90 || angleB > 90 || angleC > 90 ){
       return "Obtuse-Angled";
```

```
    }
    else if (angleA == 90 || angleB == 90 || angleC == 90){
        return "Others";
    }
    else{
        return "Acute-Angled";
    }
}
```

Figure 4: Modified ILOB code

The figure above shows the modified code for the triangle class. To achieve the goal of the activity, such as computing the perimeter, area, and identifying if the triangle is an acute, obtuse, or “other” triangle. To compute the perimeter and area, I added the attributes for the 3 sides of the triangle so that the user can set this to their liking. The perimeter calculation is easy enough, as it only needs to add the 3 different sides of the triangle. To calculate the area, it is commonly known that the formula is ½ base x height. But in order not to add another attribute, I searched for different ways to calculate the area of a triangle. I found “Heron’s Formula” for computing the area of a triangle using only its sides. I implemented it in my code with the help of the cmath module. On the other hand, to check the type of triangle based on its angle, I used an if-else-if statement. If the triangle has one angle greater than 90, then it's an obtuse triangle. If it has an angle equal to 90, it is labeled as “others”. Lastly, if the conditions are not met, it is considered an acute triangle.

```
❯ cd "/home/nielc/Downloads/HW # 1.1/" && g++ triangle.cpp -o triangle
&& "/home/nielc/Downloads/HW # 1.1/"triangle
area is 173.205
perimeter is 60
is an Obtuse-Angled triangle
The shape is a valid triangle.
```

Figure 4.1: Figure 4 code output

Conclusion:

In this lab activity, I was able to recall my learnings and learn about the basics of OOP in C++. The basics of C++, such as the code structure, were already taught to us in the last year of our program, so I only needed to recall what I had learned last time. What I found new is the way OOP works, or how classes work in C++. In this language, the syntax of creating a class is different from Python. Creating a constructor and attributes is not the same as the way it was in Python, but the concept is fairly the same. Overall, the procedure is easy enough to do. The supplementary activity also helped me to apply this remembered knowledge. The programs were easy to do as they are very simple programs that only rely on mostly one math formula. I was able to apply my learning in my math subjects to programming in the tasks on the supplementary activities. Learning how to create classes and objects in C++ is very interesting, and I am looking forward to using this concept in the future.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **It was only used to correct the grammar of the code discussion. It was not used to generate anything.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.